

Barman (Backup and Recovery Manager for PostgreSQL): A Deep Technical Report

TL;DR

- Barman is a ~100%-Python disaster-recovery tool for PostgreSQL (GPL-3.0, maintained by EnterpriseDB, latest release 3.19.1 on 26 May 2026), architected around a `(Server) → (BackupManager) → pluggable (BackupExecutor)/(WalArchiver)/(RecoveryExecutor)` object model that orchestrates two backup transports (rsync/SSH and native `(pg_basebackup)` streaming) plus two WAL-ingest paths (`(archive_command)` and `(pg_receivewal)` streaming).
- It implements a broad CLI surface (backup, restore/recover, list-backups, check, diagnose, cron, switch-wal, archive-wal, receive-wal, get-wal, put-wal, keep, config-switch/config-update, generate-manifest, verify-backup, sync-info/sync-backup/sync-wals, etc.) plus a server-less `(barman-cloud-*)` client suite for S3/Azure/GCS object storage and cloud disk snapshots.
- Recent 3.x work centers on PostgreSQL 17 block-level incremental backup (`(pg_combinebackup)` + WAL summarizer), cloud snapshot backups, WORM/immutable-storage support, config models, and GPG backup/WAL encryption; for an expert operator the key architectural takeaway is that Barman cleanly separates *transport strategy* from *catalog/metadata management*, which is what makes RPO≈0 (synchronous `(pg_receivewal)`) and flexible PITR possible.

Key Findings

- Language/runtime:** Python (the GitHub repo reports Python 100.0%), minimum supported Python 3.8 (3.6/3.7 deprecated as of 3.14), requiring `(psycopg2 >= 2.4.2)`, `(python-dateutil)`, `(setuptools)`; optional extras pull in `(boto3)`, `(azure-identity)`/`(azure-storage-blob)`/`(azure-mgmt-compute)`, `(google-cloud-storage)`/`(google-cloud-compute)`, `(zstandard)`, `(lz4)`, `(python-snappy)`/`(cramjam)`, and `(argcomplete)`. PostgreSQL server ≥ 13 and `(rsync >= 3.1.0)` are the baseline external dependencies. [GitHub](#) [GitHub](#)
- Design philosophy:** A single Barman host centrally manages many PostgreSQL servers remotely over the network, never sharing physical resources with the DB host. It layers on PostgreSQL's native PITR primitives (base backup + WAL replay) rather than reinventing them, and exposes a strategy/executor architecture so the same catalog logic serves rsync, streaming, snapshot, and cloud backends.
- Two backup transports:** `(backup_method = rsync)` (SSH transport, supports file-level incremental via hard-links/`(reuse_backup)` and parallel copy) vs `(backup_method = postgres)` (native `(pg_basebackup)` streaming, supports PG17 block-level incremental, server-side compression). Additional methods: `(snapshot)` (cloud disk snapshots) and `(local-to-cloud)` (experimental).
- Two WAL paths:** `(archiver)` (PostgreSQL `(archive_command) → (barman-wal-archive)/(put-wal)` or rsync into `(incoming/)`) and `(streaming_archiver)` (`(pg_receivewal)` into `(streaming/)` with `(.partial)` files), the latter enabling RPO→0 including synchronous standby registration.
- Catalog & consistency:** per-server `(xlog.db)` WAL catalog + per-backup `(backup.info)` metadata; backups pass through STARTED → WAITING_FOR_WALS → DONE (or FAILED) states, with `(check-backup)` enforcing WAL continuity from backup begin to end LSN.
- Recovery:** `(recovery_executor_factory)` dispatches to `(RecoveryExecutor)`, `(TarballRecoveryExecutor)` (compressed), `(IncrementalRecoveryExecutor)` (PG17 chains via `(pg_combinebackup)`), or `(SnapshotRecoveryExecutor)`; PITR targets time/LSN/XID/name/immediate/timeline; recovery config is written to `(recovery.conf)` (pre-PG12) or `(postgresql.auto.conf)` + `(recovery.signal)`/`(standby.signal)` (PG12+).

Details

1. Overview, history, and design philosophy

Barman ("Backup and Recovery Manager") is an open-source administration tool for disaster recovery of PostgreSQL servers, written in Python and distributed under GNU GPL 3. It was created by 2ndQuadrant — per the Barman Manual, "In 2011, with these goals in mind, 2ndQuadrant started the development of Barman, now one of the most used backup tools for PostgreSQL. Barman is an acronym for 'Backup and Recovery Manager.'" It is now maintained by EnterpriseDB (EDB), which acquired 2ndQuadrant on 30 September 2020 (EDB's press release: "EDB, a leading contributor to PostgreSQL, today announced that it has acquired 2ndQuadrant, a global PostgreSQL solutions and tools company based out of the UK"). Since version 2.13 the canonical home is the GitHub repository `(EnterpriseDB/barman)` (replacing the legacy SourceForge repository); the repo shows Python at 100.0% of the codebase and carries a GPL-3.0 license with copyright "2011-2025 EnterpriseDB UK Limited." The latest release is 3.19.1, confirmed verbatim on the pgbarman.org homepage as "Version 3.19.1 (26 May 2026)"; the preceding feature release 3.19.0 added cloud-storage restore support. [GitHub](#)

The core design premise: a dedicated Barman server, physically separate from the PostgreSQL host(s) (sharing nothing but the network), centrally manages remote backups of many PostgreSQL servers. Barman deliberately builds on PostgreSQL's own robust PITR technology — physical base backup plus continuous WAL archiving and replay — rather than implementing block-level mechanics itself (until PG17 delivered native incremental backup, which Barman then wraps). Architecturally, Barman separates *transport strategy* (how bytes move: rsync/SSH vs streaming replication protocol) from *catalog and metadata management* (the backup catalog, WAL catalog, retention, checks).

This is why the same higher-level workflows serve four backup methods. Barman's own guidance (3.12 architecture docs) is explicit about the trade-off: "for general usage we recommend using streaming replication for all currently supported versions of Postgres... Because Barman makes use of pg_basebackup when using streaming backups, features such as parallel backup are currently not available."

2. Language, codebase architecture, and dependencies

Language/runtime. Pure Python. Minimum supported Python is 3.8 (support for 3.6 and 3.7 was deprecated as of Barman 3.14; 3.14 is known not to work on 3.6). The install baseline is PostgreSQL server ≥ 13 , PostgreSQL client tools, and `(rsync >= 3.1.0)` (required for recovery and rsync backups).

Dependencies (from `setup.py`). Mandatory `(install_requires)`: `(psycopg2 >= 2.4.2)` (PostgreSQL connectivity), `(python-dateutil)`, and `(setuptools)` (because Python 3.12+ no longer bundles it). The `(extras_requires)` map defines optional feature groups: `(argcomplete)`; `(aws-snapshots) → (boto3)`; `(azure) → (azure-identity, azure-storage-blob)`; `(azure-snapshots) → (azure-identity, azure-mgmt-compute)`; `(cloud) → (boto3)`; `(google) → (google-cloud-storage)`; `(google-snapshots) → (grpcio, google-cloud-compute)`; `(snappy) → (python-snappy, cramjam >= 2.7.0)`; `(zstandard)`; `(lz4)`. An `(all)` extra is auto-generated by unioning every group. Historically (Barman 1.x/early 2.x) Barman depended on `(argh)` for CLI dispatch; the modern codebase uses `(argparse)` directly (the `(barman.cli)` module shows `(argparse.ArgumentParser)`/`(HelpFormatter)`). `(boto3 >= 1.29.1)` is specifically required for AWS snapshot lock. [\(GitHub\)](#)

Project layout (repo root). `(barman/)` (Python sources), `(docs/)` (tutorial and man pages), `(scripts/)` (auxiliary scripts), `(tests/)` (unit tests), plus `(setup.py)`, `(tox.ini)`, `(requirements-tox.txt)`, `(README.rst)`, `(RELNOTES.md)`, `(AUTHORS)`, `(INSTALL.md)`, `(LICENSE)`, and lint configs `(.isort.cfg)`, `(.python-black)`, `(.markdownlint.yml)`, `(.yamllint.yml)`, `(.hadolint.yaml)`, `(.gitleaks.toml)`. The repo has 2,563 commits and 39 releases as of this writing.

Entry points (`(console_scripts)` in `(setup.py)`). Twelve CLI executables are installed: `(barman)` (`(barman.cli:main)`), `(barman-cloud-backup)`, `(barman-cloud-wal-archive)`, `(barman-cloud-restore)`, `(barman-cloud-wal-restore)`, `(barman-cloud-backup-delete)`, `(barman-cloud-backup-keep)`, `(barman-cloud-backup-list)`, `(barman-cloud-backup-show)`, `(barman-cloud-check-wal-archive)` (each `(barman.clients.cloud_*.main)`), and `(barman-wal-archive)` / `(barman-wal-restore)` (`(barman.clients.walarchive:main)` / `(walrestore:main)`). The version is read from `(barman/version.py)`. [\(GitHub\)](#) [\(GitHub\)](#)

3. Module-by-module breakdown

- `(barman/cli.py)`** — the `(barman)` command dispatcher. Built on `(argparse)` (functions decorated with a `(command)` decorator; `(argument(*name_or_flags, **kwargs))` helper formats arguments). Contains `(main())`, `(manage_server_command())` (standard handling of inactive/disabled servers), backup-ID alias parsing (`(latest/last)`, `(oldest/first)`, etc.), and one function per subcommand (`(backup)`, `(recover)`, `(restore)`, `(check)`, `(check_backup)`, `(receive_wal)`, `(get_wal)`, `(put_wal)`, `(replication_status)`, `(sync_info)`, `(rebuild_xlogdb)`, etc.).
- `(barman/server.py)`** — the `(Server)` class ("Barman is able to manage multiple servers"). Owns the PostgreSQL connections (`(self.postgres)`, streaming connection), instantiates archivers (`(FileWalArchiver)`, `(StreamingWalArchiver)`), holds the `(BackupManager)`, and drives `(check)`, `(cron)`, `(switch_wal)`, `(replication_status)`, `(get_backup)`, and standby-awareness logic (`(pg_is_in_recovery())`). Also defines `(CheckStrategy)`/`(CheckOutputStrategy)` (the check-collection framework with the `(CheckResult)` namedtuple) and sync constants (`(PRIMARY_INFO_FILE)`, `(SYNC_WALS_INFO_FILE)`, `(PARTIAL_EXTENSION = ".partial")`).
- `(barman/backup.py)`** — `(BackupManager(RemoteStatusMixin, KeepManagerMixin))`, "Manager of the backup archive for a server." Coordinates the backup lifecycle, retention enforcement, backup deletion, `(check_backup)`, `(verify_backup)` (invokes `(pg_verifybackup)`), catalog navigation (`(get_previous_backup)`, `(get_backup)`), and constructs the appropriate executor. Defines status filters (`(DEFAULT_STATUS_FILTER = BackupInfo.STATUS_COPY_DONE)`), a `(DELETE_ANNOTATION = "delete_this")` marker, and imports the `(recovery_executor_factory)`.
- `(barman/backup_executor.py)`** — the strategy/executor hierarchy. Abstract `(BackupExecutor)` is invoked by `(BackupManager.backup())`; `(ExternalBackupExecutor)` performs backups over SSH (remote mode default) or locally and is a factory for exclusive/concurrent strategy objects; `(RsyncBackupExecutor)` copies via rsync; `(PostgresBackupExecutor)` drives `(pg_basebackup)`; `(SnapshotBackupExecutor)` creates cloud disk snapshots (no data-file copy) via a `(CloudSnapshotInterface)`; `(PassiveBackupExecutor)` is the dummy executor for passive/geo-redundant nodes (requires `(primary_ssh_command)`). Backup strategies: `(ExclusiveBackupStrategy)` (legacy `(pg_start_backup)`/`(pg_stop_backup)`), `(ConcurrentBackupStrategy)`/`(LocalConcurrentBackupStrategy)` (concurrent backup API), with `(START_TIME_RE)` and related parsing. `(Pgbarman + 4)`
- `(barman/recovery_executor.py)`** — recovery orchestration. `(recovery_executor_factory(backup_manager, command, backup_info))` returns `(IncrementalRecoveryExecutor)` (if `(backup_info.is_incremental)`), `(SnapshotRecoveryExecutor)` (if `(snapshots_info)`), or, based on compression, `(RecoveryExecutor)` (none) / `(TarballRecoveryExecutor)` (`(gzip/lz4/zstd)`). Handles `(set_pitr_targets())`, `(_generate_recovery_conf())`, archive-status generation, and PG17 chain combination via `(PgCombineBackup)`. Imports `(RsyncCopyController)`, compression classes, and `(SyntheticBackupInfo)`.

- `barman/wal_archiver.py` — `WalArchiver` base with `FileWalArchiver` (processes `incoming/` filled by `archive_command`), `StreamingWalArchiver` (manages `pg_receivewal`, sets `application_name = barman_receive_wal`, handles `.partial`), a `WalArchiverQueue`/batch abstraction (`archiver_batch_size`), and a cloud WAL archiver for `pg_wal`-resident WALs.
- `barman/copy_controller.py` — `RsyncCopyController` (parallel rsync engine: `workers`, `workers_start_batch_period/size`, `reuse_backup`, `safe_horizon` for checksum-skip optimization, `bwlimit`, `VANISHED_RE` to tolerate rsync exit code 23) and the internal `_RsyncCopyItem` describing each copy unit; `reuse` implements incremental copy via hard links.
- `barman/infofile.py` — the metadata layer. `FieldListFile`/`Field` descriptors serialize key/value metadata; `BackupInfo`/`LocalBackupInfo` model `backup.info` (with statuses `EMPTY/STARTED/WAITING_FOR_WALS/DONE/FAILED`, chain-walk helpers `walk_to_root`, `is_incremental`, checksum-consistency checks); `SyntheticBackupInfo` represents a `pg_combinebackup` output; `VolatileBackupInfo` holds in-memory overrides (e.g., during decrypt/combine); `WalFileInfo.from_file(...)` builds WAL metadata and `to_xlogdb_line()` serializes an `xlog.db` line.
- `barman/xlog.py` — WAL name/segment arithmetic (`decode_segment_name`, `hash_dir`, `is_history_file`, `is_any_xlog_file`, segment-size handling) underpinning the WAL catalog and continuity checks.
- `barman/cloud.py` — the cloud abstraction: abstract `CloudInterface` (multipart/async uploads via worker processes and a job queue; `async_upload_part`, `upload_part`, `complete_multipart_upload`, `remote_open`, `download_file`, `delete_objects`), `CloudUploadController`/`CloudTarUploader` (tar chunking of backups), and `CloudBackupCatalog`. Provider implementations live in `barman/cloud_providers/` (`aws_s3.py` → `S3CloudInterface`, `azure_blob_storage.py` → `AzureCloudInterface`, `google_cloud_storage.py` → `GoogleCloudInterface`), plus snapshot interfaces and `SnapshotMetadata`.
- `barman/clients/` — the standalone `barman-cloud-*` and `barman-wal-*` scripts (`cloud_backup.py`, `cloud_restore.py`, `cloud_walarchive.py`, `cloud_walrestore.py`, `cloud_backup_delete.py`, `cloud_backup_keep.py`, `cloud_backup_list.py`, `cloud_backup_show.py`, `cloud_check_wal_archive.py`, `walarchive.py`, `walrestore.py`, `cloud_cli.py`). `cloud_cli.py` provides `CloudArgumentParser` (subclass of `argparse.ArgumentParser`, exits with `CLIBErrorExit`/code 3) and provider inference from the destination URL.
- `barman/encryption.py` — `EncryptionManager` (validates config, dispatches via a `REGISTRY = {'pgpg': (GPGEncryption, ...)}}`), abstract `Encryption`, and `GPGEncryption` (methods `encrypt`, `decrypt`, `recognize_encryption`); plus `get_passphrase_from_command()`. Introduced in Barman 3.14. `Pgbarman` `Pgbarman`
- **Supporting modules:** `barman/config.py` (INI parsing, `ServerConfig`, `ModelConfig`, `RecoveryOptions`), `barman/hooks.py` (`HookScriptRunner`, `RetryHookScriptRunner`), `barman/retention_policies.py`, `barman/command_wrappers.py` (wrappers for `pg_basebackup`, `pg_receivewal`, `pg_verifybackup`, `PgCombineBackup`, `RsyncPgData`), `barman/fs.py` (`UnixLocalCommand`), `barman/process.py` (subprocess/PID management), `barman/lockfile.py` (`ServerBackupIdLock`, `ServerBackupSyncLock`, etc.), `barman/output.py` (console/JSON/Nagios writers), `barman/diagnose.py`, `barman/annotations.py` (keep/delete annotations), and `barman/remote_status.py`.

4. Command catalog

Core `barman` server/global commands:

- `backup SERVER [...]` — take a base backup; options include `--incremental BACKUP_ID`, `--reuse-backup {off|copy|link}`, `--immediate-checkpoint`/`--no-immediate-checkpoint`, `-j/--jobs`, `--jobs-start-batch-period/size`, `--name`, `--manifest`/`--no-manifest`, `--wait`/`--wait-timeout`, `--retry-times`/`--retry-sleep`, `--bwlimit`. (all) backs up every server.
- `restore` (renamed from `recover` in 3.x; `recover` retained as alias) — restore to a local or remote (`--remote-ssh-command`) directory with PITR targets `--target-time`, `--target-lsn`, `--target-xid`, `--target-name`, `--target-immediate`, `--target-tli`, plus `--target-action {pause|shutdown|promote}`, `--standby-mode`, `--get-wal`/`--no-get-wal`, `--local-staging-path` (incremental combine), `--recovery-staging-path` (compressed), `--recovery-conf-filename`, `--tablespace NAME:LOCATION`, snapshot recovery options.
- `list-backups` / `list-backup`, `show-backup`, `list-servers` / `show-servers`, `show-server`, `status`, `list-processes`, `terminate-process`, `list-files`.
- `check` (with `--nagios`), `diagnose` (`--show-config-source`) — JSON diagnostic dump of global config, SSH/Python/rsync versions, and all servers.
- `cron` — maintenance orchestrator (`--keep-descriptors`).
- `switch-wal` (`--force`, `--archive`, `--archive-timeout`; renamed from `switch-xlog` in 2.3).
- `archive-wal`, `receive-wal` (`--create-slot`, `--drop-slot`, `--stop`, `--reset`, `--test`), `get-wal` (`--peek`, `--test`, `--partial`), `put-wal` (SSH endpoint for `barman-wal-archive`).
- `delete` (`--dry-run`), `keep` (`--target {full|standalone}`, `--status`, `--release`), `check-backup`, `verify-backup` (runs `pg_verifybackup`), `generate-manifest`, `rebuild-xlogdb`.

- `replicate/replication-status` (live streaming client status).
- `config-switch SERVER {MODEL|--reset}`, `config-update JSON_CHANGES` (writes `.barman.auto.conf`).
- Geo-redundancy: `sync-info [--primary] SERVER [last_wal [last_position]]`, `sync-backup`, `sync-wals`.

barman-cloud-* and helper scripts (server-less / DB-host-side):

- `barman-cloud-backup` (backup local PG → object storage tarballs, or take cloud snapshots), `barman-cloud-restore`, `barman-cloud-wal-archive` (used directly in `archive_command`), `barman-cloud-wal-restore` (used as `restore_command`), `barman-cloud-backup-delete` (`-r RETENTION_POLICY`, `--check-object-lock`), `barman-cloud-backup-keep`, `barman-cloud-backup-list`, `barman-cloud-backup-show`, `barman-cloud-check-wal-archive`.
- `barman-wal-archive` and `barman-wal-restore` — the barman-cli helpers that talk to the Barman server's `put-wal`/`get-wal` over SSH (tar-stream with MD5SUMS validation).

5. Workflow pathways

Backup workflow. `BackupManager.backup()` invokes the configured `BackupExecutor`. The strategy's `start_backup()` calls the PostgreSQL low-level API and records `begin_wal`/`begin_xlog`/`begin_offset` into `backup.info` (which is `save()`d immediately), logging "Backup start at LSN...". Then `backup_copy()` runs the transport, `stop_backup()` records end LSN, and the backup transitions to `WAITING_FOR_WALS` until continuity is confirmed.

- *rsync/SSH* (`backup_method = rsync`): `RsyncBackupExecutor` + `RsyncCopyController` copy PGDATA and tablespaces over SSH, honoring `parallel_jobs`, `bandwidth_limit`/`tablespace_bandwidth_limit`, and `reuse_backup`. `reuse_backup = link` creates hard links to unchanged files in the previous backup (file-level dedup); `copy` re-copies unchanged files (time-only saving). This is the only method supporting file-level incremental/dedup and `--jobs` parallel copy.
- *Streaming* (`backup_method = postgres`): `PostgresBackupExecutor` drives `pg_basebackup` over the replication protocol. Supports server-side `backup_compression` (`gzip`/`lz4`/`zstd`/`none`) and PG17 block-level incremental via `--incremental` (requires `summarize_wal = on` and the WAL summarizer; each incremental references a prior full/incremental backup by ID). `rsync`-style dedup and `--jobs` are not available here. `Pgbarman`
- *Backup API history*: `ExclusiveBackupStrategy` uses the deprecated exclusive `pg_start_backup()`/`pg_stop_backup()`; `ConcurrentBackupStrategy`/`LocalConcurrentBackupStrategy` use the concurrent (non-exclusive) API and write the `backup_label` on the Barman side. `Pgbarman`
- *Snapshot* (`backup_method = snapshot`): no data-file copy; `SnapshotBackupExecutor` creates provider disk snapshots via `CloudSnapshotInterface`, verifies the target disks are attached/mounted, records mount points/options in `backup.info`, and still stores `backup_label` (concurrent) and `backup.info` locally. `Pgbarman`
- *Metadata*: `backup.info` records begin/end WAL and offsets, timeline, `mode` (e.g., `rsync-concurrent`, `postgres`), `size/deduplicated_size`, `copy_stats`, tablespaces, `systemid`, `xlog_segment_size`, and status.

WAL archiving workflow. Two ingestion paths, at least one required:

- *archive_command path* (`archiver = on`): PostgreSQL runs `archive_command`, either `rsync` into the server's `incoming/` directory, or `barman-wal-archive HOST SERVER %p`, which SSHes to the Barman `put-wal` endpoint (tar stream + MD5SUMS validation). A recommended safeguard: `test $(hostname --fqdn) = HOSTNAME && rsync ...` to avoid ingesting WAL from a cloned server.
- *streaming path* (`streaming_archiver = on`): `StreamingWalArchiver` runs `pg_receivewal` against a replication slot (`slot_name`, `create_slot = auto`), writing `.partial` files into `streaming/`; completed segments lose `.partial` and are handed to `archive-wal`. This can drive RPO→0, including registering Barman as a synchronous standby (`synchronous_standby_names = 'barman_receive_wal'`). `Pgbarman`
- *Catalog*: the per-server `xlog.db` records one line per WAL (name, size, timestamp, compression), e.g. `000000020002E59C000000BC3089819 1615159937.46 pybzip2`. `archive-wal` applies compression (`gzip`/`bzip2`/`pigz`/`pygzip`/`pybzip2`/`lz4`/`zstd`/`custom`) and moves segments into the hashed archive. `rebuild-xlogdb` regenerates `xlog.db` from disk. `max_incoming_wals_queue` bounds the incoming queue.

Recovery/restore workflow. `recovery_executor_factory` selects the executor. `setup()` builds a recovery map, `set_pitr_targets()` decides `is_pitr` (true if any target time/xid/lsn/name/immediate/non-default timeline is set), and `generate_recovery_conf()` writes recovery configuration — `recovery.conf` for pre-PG12, or appends to `postgresql.auto.conf` and touches `recovery.signal` (or `standby.signal` for `--standby-mode`) for PG12+ (filename overridable with `--recovery-conf-filename`). WAL supply during recovery is either a full copy of required WALs or, with `get-wal` in `recovery_options`, a generated `restore_command` using `barman get-wal` (local) or `barman-wal-restore` (remote) — turning Barman into a WAL hub. Remote recovery uses `--remote-ssh-command`. For PG17 incremental

backups, `IncrementalRecoveryExecutor` assembles the chain (root full + all intermediate incrementals) into a synthetic full backup with `pg_combinebackup` in `local_staging_path`, then places it at the destination. Compressed backups are staged/uncompressed via `recovery_staging_path` (`TarballRecoveryExecutor`). Requirements: matching PG major version and identical hardware architecture between source and restore host.

Cron/maintenance workflow. `barman cron` (recommended every minute) is the maintenance engine. It starts the streaming `receive-wal` process when needed, processes accumulated WALs (`archive-wal`, compression), enforces retention policies, runs `check-backup` to promote WAITING_FOR_WALS backups to DONE (or FAILED if the first required WAL never arrives), completes any backups left with a delete marker after a crash, and (for passive nodes) triggers `sync-info`/`sync-backup`/`sync-wals`. Coordination uses lock files in `barman_lock_directory` (default `barman_home`; a volatile path like `/var/run/barman` is recommended). `config-update` requests are serialized via the `cfg_changes.queue`.

Geo-redundancy workflow. Introduced in Barman 2.6 (PostgreSQL.org announcement: "This release introduces support for geographical redundancy... Geo-redundancy allows a Barman server to use another Barman server as a data source, instead of using a PostgreSQL server. Add 'put-wal' command..."). A *passive node* sets `primary_ssh_command = ssh <primary-barman>` (instead of connecting to PostgreSQL); its backup source is another Barman installation. `barman cron` transparently runs `sync-info --primary` (poll JSON: completed backups, archived WALs, config, last `xlog.db` position), then `sync-backup` and `sync-wals` to rsync backups/WALs from the primary. Because it reuses the rsync engine, geo-redundancy inherits parallel copy and network compression. Passive nodes forbid direct PostgreSQL operations (including `barman backup`) and can carry their own (typically longer) retention policy — enabling multi-tier / cross-site DR.

Cloud workflows. The `barman-cloud-*` tools operate without a Barman server, driven by `CloudInterface` subclasses (`S3CloudInterface` via boto3, `AzureCloudInterface` via the Azure SDK, `GoogleCloudInterface` via google-cloud-storage). `barman-cloud-backup` streams `pg_basebackup` output into tar chunks (`CloudTarUploader`/`CloudUploadController`) uploaded via multipart/async workers; `-J/--jobs`, `-S/--max-archive-size`, and compression (`-z`/`-j`/`--snappy`) control it. `barman-cloud-wal-archive` slots into `archive_command`; `barman-cloud-wal-restore` into `restore_command`. Cloud snapshot backups (AWS/Azure/GCP) require `findmnt`, managed disks, and provider libraries; snapshot metadata carries provider/region/device/snapshot IDs and mount points. Retention without a server is enforced via `barman-cloud-backup-delete -r`. The tools can also run as Barman hook scripts (`post_backup_retry_script = barman-cloud-backup ...`), reading `BACKUP_DIR`/`BACKUP_ID` from the environment. A `local-to-cloud` backup method also exists (experimental; S3 only; no incremental/dedup; restore not yet supported). [DeepWiki + 3](#)

6. Configuration model, checks, hooks, encryption

Configuration. Plain INI. A global `[barman]` section (`barman_home`, `barman_user`, `configuration_files_directory` default `/etc/barman.d`, `log_file`, `log_level`, `barman_lock_directory`, `compression`, etc.) plus one `[servername]` section per PostgreSQL server. Per-server keys include `description`, `conninfo`, `streaming_conninfo`, `wal_conninfo`/`wal_streaming_conninfo` (dedicated WAL-monitoring/streaming connections), `ssh_command`, `backup_method`, `archiver`, `streaming_archiver`, `slot_name`/`create_slot`, `retention_policy`, `minimum_redundancy`, `reuse_backup`, `parallel_jobs`, `path_prefix`, `primary_conninfo`/`primary_checkpoint_timeout` (standby backup), `last_backup_maximum_age`, and directory overrides (`basebackups_directory`, `wals_directory`, `incoming_wals_directory`, `streaming_wals_directory`, `xlogdb_directory`). Options carry scope tags (Global / Server / Model). Setting `active = false` when adding a server is best practice until `barman check` is clean.

Config models (Barman 3.10+). A third config type: a *model* is a named set of overrides scoped to a `cluster`. `model = true` turns a section into a model; `cluster` groups servers/models. `barman config-switch SERVER MODEL` applies overrides (only one model active per server at a time; `--reset` clears them) — used to redirect streaming/archiving to a new primary after failover. `barman config-update JSON_CHANGES` creates/updates servers or models from JSON, writing to `.barman.auto.conf` in `barman_home` (higher precedence than `/etc/barman.conf` and included files).

Check framework & diagnostics. `barman check` (backed by `CheckStrategy` in `server.py`) runs an extensive battery: PostgreSQL connectivity, superuser/backup privileges, streaming, `wal_level`, replication slot, directories, retention policy settings, backup maximum age, compression settings, failed backups, minimum redundancy, `pg_basebackup`/`pg_receive_xlog` presence and compatibility, `archive_mode`/`archive_command`, continuous archiving, `receive-wal running`, `systemid coherence`, and `archiver errors`. It is intended for integration with monitoring (`--nagios` yields a Nagios/Icinga plugin). `barman diagnose` emits a JSON snapshot for support (format explicitly not a stable interface). There is no built-in Prometheus exporter; monitoring is typically layered on `check --nagios`, `status`, `replication-status`, and `diagnose` JSON.

Hook scripts. Managed by `HookScriptRunner`/`RetryHookScriptRunner` in `barman/hooks.py`. Standard hooks run once (return code ignored); retry hooks re-run until they return SUCCESS (0), ABORT_CONTINUE (62), or ABORT_STOP (63) — `RetryHookScriptRunner` defines `ATTEMPTS_BEFORE_NAP = 5`, `BREAK_TIME = 3`, `NAP_TIME = 60`. Hook points: `pre/post_backup[_retry]_script`, `pre/post_archive[_retry]_script`, `pre/post_delete[_retry]_script`, and recovery hooks (`pre/post_recovery[_retry]_script`; delete/recovery hooks added in 2.4). Execution order: standard-pre → retry-pre → main event (unless retry-pre returned ABORT_STOP) →

retry-post → standard-post. The environment carries `BARMAN_PHASE` (pre/post), `BARMAN_SERVER`, `BARMAN_CONFIGURATION`, `BARMAN_ERROR`, and event-specific vars: backup (`BARMAN_BACKUP_ID`, `BARMAN_BACKUP_DIR`, `BARMAN_BACKUP_INFO_PATH`, `BARMAN_PREVIOUS_ID`, `BARMAN_STATUS`, `BARMAN_RETRY`, `BARMAN_VERSION`), WAL (`BARMAN_SEGMENT`, `BARMAN_FILE`, `BARMAN_SIZE`, `BARMAN_TIMESTAMP`, `BARMAN_COMPRESSION`), delete (`BARMAN_NEXT_ID`), recovery (`BARMAN_DESTINATION_DIRECTORY`, `BARMAN_TABLESPACES`, `BARMAN_REMOTE_COMMAND`, `BARMAN_RECOVER_OPTIONS`).

Retention & consistency. `barman/retention_policies.py` defines `RedundancyRetentionPolicy` (`REDUNDANCY N`), `RecoveryWindowRetentionPolicy` (`RECOVERY WINDOW OF N [DAYS|WEEKS|MONTHS]`), and `SimpleWALRetentionPolicy` (`wal_retention_policy = MAIN`), created via `RetentionPolicy.create(...)`. Backups are labeled VALID / OBSOLETE / and (via `keep`) archival; `minimum_redundancy` protects a floor of full backups (fixed in 3.x to count only non-incremental backups). For PG17 incremental chains, only the root backup is evaluated for retention and its label propagates to children (e.g., an OBSOLETE root makes children OBSOLETE; `KEEP:FULL` root keeps children VALID). Consistency is enforced by the STARTED→WAITING_FOR_WALS→DONE state machine plus `check-backup`, which requires every WAL from backup begin to end LSN to be archived before marking DONE (missing first WAL ⇒ FAILED). `keep --target full` retains all WALs (and child backups) for latest-point PITR; `keep --target standalone` retains only WALs needed to reach the backup's own consistency point.

Encryption (Barman 3.14+). `barman/encryption.py` implements GPG (GnuPG ≥ 2.1) asymmetric encryption of `pg_basebackup` tar backups and WAL files. Config options: `encryption` (`gpg` or `none`), `encryption_key_id` (public key ID/recipient), and `encryption_passphrase_command`; encrypted files get a `.gpg` extension and the pipeline is Source→Compress→Encrypt→Storage (encryption requires compression). Notably, because encryption requires compression, incremental backups initially could be taken with *neither* — this was resolved in Barman 3.16.x (BAR-764/801/841): "Barman now fully supports taking incremental backups with both compression and/or encryption... Previously, Barman prevented incremental backups to be taken if compression was enabled or if the parent backup was already compressed. Since encryption in Barman requires compression, this also meant that incremental backups could not be taken with encryption." `barman check` validates encryption settings and `show-backup` reports whether/how a backup was encrypted; decryption during restore is staged locally. Note the distinction: the GPG path applies to server-side `barman backup`/`archive-wal`, whereas the `barman-cloud-*` tools use provider server-side encryption instead — `--encryption {AES256|aws:kms}` with `--sse-kms-key-id` for S3, and `--kms-key-name` for GCS/KMS. (Safety DB + 4)

WORM / immutable storage. Barman supports storing backups on WORM (Write-Once-Read-Many) immutable storage for ransomware protection, with `worm_mode` and relocation of the mutable `backup.info` into a `meta/` directory inside the backup directory; keep/delete annotations were moved to a dedicated metadata directory so they still work when the backup body is immutable. Guidance: only `basebackups_directory` and `wals_directory` should be immutable, while `xlogdb_directory`, staging paths, and internal metadata stay on regular storage; `barman keep` is unsupported in immutable environments. AWS EBS Snapshot Lock (compliance/governance modes) provides equivalent WORM protection for cloud snapshots. (Pgbarman) (Pgbarman)

7. Recent evolution (3.x highlights)

- PG17 block-level incremental backup** (Barman 3.11): first tool to wrap PostgreSQL 17 native incremental backup. The PostgreSQL.org "Barman 3.11.1 and 3.11.0" announcement lists: "Add support for Postgres 17+ incremental backups... Add `--incremental` command-line option to `barman backup`... `latest-full` shortcut backup ID," and the docs note "Starting with version 3.11, Barman also supports block-level incremental backups through the streaming connection." It requires `summarize_wal = on` and `backup_method = postgres`; restore combines chains via `pg_combinebackup`. Barman 3.11 also added the "Resources saving" metric to `show-backup` — an estimate of resources saved by incremental vs. full-cluster backup (the underlying field was renamed from `incremental_size` to `resource_savings`).
- Config models & `config-update`** (3.10): JSON-driven config and cluster-scoped overrides for HA failover redirection.
- Cloud snapshot backups** across AWS/Azure/GCP; AWS EBS Snapshot Lock and S3 Object Lock (`--check-object-lock`) for immutability.
- WORM/immutable support** and crash-safe deletion via delete markers processed by cron.
- GPG encryption** of backups and WALs (3.14); compression+encryption for incrementals unblocked in 3.16.x.
- Terminology change:** `barman recover` renamed to `barman restore` (the docs and code now distinguish "restore" from "recover/recovery"); `switch-xlog` → `switch-wal` earlier.
- PG18 forward-looking:** EDB engineers describe a planned `pg_combinebackup --link` optimization (contributed by Israel Barth Rubio) to make incremental restore cheaper; Barman is planned to adopt it — this is described as a future enhancement, not a shipped Barman feature. (EnterpriseDB)

Testing & packaging. Tests live in `tests/` and run under `pytest` orchestrated by `tox` (`tox.ini`, `requirements-tox.txt`); the codebase uses black/isort/markdownlint/yamllint/hadolint. Distribution: PyPI (`pip install barman`), building psycopg2 needs libpq headers + gcc, and RPM/DEB via the PostgreSQL Global Development Group (PGDG) repositories — packages `barman`, `barman-cli`, and `barman-cli-`

`cloud` (the latter bundled into `barman-cli` on RHEL). EPEL is required on RHEL/Rocky/Alma. License is GPL-3.0; maintainer is EnterpriseDB. `Pgbarman` `Pgbarman`

Recommendations

1. **Default to `backup_method = postgres` + `streaming_archiver = on` for new deployments** unless you need rsync-only capabilities. Streaming is easier to set up, supports PG17 block-level incrementals, and (with a replication slot + synchronous registration) achieves RPO≈0. **Switch to (or add) `backup_method = rsync` when:** you have a very large database that benefits from file-level dedup/hard-link incrementals (`reuse_backup = link`) and parallel copy (`parallel_jobs`), which the postgres method does not offer.
2. **Instrument `barman check --nagios` and `barman cron` (every minute) in monitoring/alerting first, before relying on backups.** The check framework (especially "receive-wal running", "continuous archiving", "archiver errors", and "backup maximum age" via `last_backup_maximum_age`) is the earliest warning that RPO is silently degrading. Treat a WAITING_FOR_WALS backup that never reaches DONE as a WAL-continuity incident.
3. **For PG17 incremental chains, provision `local_staging_path` on fast storage sized for a full synthetic backup**, since `pg_combinebackup` reconstructs a complete data directory at restore time. Benchmark restore time end-to-end; if the combine step dominates RTO, keep a shorter incremental chain (more frequent fulls) or wait for the PG18 `--link` optimization before extending chains.
4. **Use config models + `config-switch` to script failover redirection** in HA clusters (repmgr/Patroni), rather than editing per-server files by hand; validate with `barman check` after each switch.
5. **For compliance/ransomware requirements, combine `worm_mode` (or S3 Object Lock / EBS Snapshot Lock) with GPG encryption**, keeping only `basebackups_directory`/`wals_directory` immutable and `xlogdb_directory`/staging on mutable storage. Set the retention window ≥ the immutability grace period, and remember `barman keep` is unsupported under WORM.
6. **Thresholds that should change your plan:** if `barman check` reports replication-slot/receive-wal failures, prioritize fixing streaming before the next backup window (backups will not start if WALs aren't being archived); if minimum-redundancy or recovery-window enforcement would delete your only full backup in an incremental chain, add a `keep` mark first.

Caveats

- Barman is Linux/Unix only; PITR restore requires the *same PostgreSQL major version* and identical hardware architecture on the restore host.
- The `postgres` (`pg_basebackup`) method does **not** support rsync-style file-level deduplication, hard-link incrementals, or `--jobs` parallel copy; conversely PG17 block-level incrementals require the `postgres` method and `summarize_wal = on`.
- `local-to-cloud` backup is experimental (S3-only, no incremental/dedup, restore not yet implemented in Barman); several Barman subcommands (`verify-backup`, `generate-manifest`, `rebuild-xlogdb`, `get-wal`) are unsupported against it.
- Asynchronous `pg_receivewal` streaming still risks losing in-flight transactions unless Barman is registered as a *synchronous* standby; the `.partial` file may hold the only copy of the latest data after a primary failure.
- `barman diagnose` JSON structure is explicitly not a stable API and may change between versions.
- The `barman-cloud-*` GPG-client-side-encryption gap (cloud tools rely on provider server-side/KMS encryption rather than GPG) is inferred from user reports and provider-encryption docs, not an unambiguous official statement; verify against the exact release you deploy.
- Version-specific details here draw on Barman 3.10–3.19 documentation; confirm option availability against the precise version you run, since options and defaults have shifted across 3.x.